Lecture 15 · Part IV · Lecture notes

Time series

Dataset: Chicago transit ridership

How to use these notes

The primary text for this lecture is Géron, Chapter 13. These notes are not a replacement for it and do not repeat it: they are the lecture's own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture's notebook, and the course checks that mechanically rather than trusting it.

Contents

1 A new kind of data **1**

1.1 The brief . . . 1

1.2 Four lines of tidying, one of which matters . . . 2

2 Derivation: stationarity, differencing and autocorrelation **3**

2.1 Differencing . . . 3

2.2 Autocorrelation, and when differencing helps . . . 3

3 Baselines, and the metric **4**

3.1 Turning one series into a table . . . 5

4 The linear model **5**

5 The protocol, and what it is worth **5**

5.1 How much of the gap is the split? . . . 6

5.2 Purging and embargo . . . 7

6 The notebook **7**

7 Exercises **7**

Summary **8**

1 A new kind of data

Everything so far assumed the rows were **exchangeable**: a random split was valid because any row was as good as any other for training.

Not here. In a time series the rows are ordered and neighbouring rows are alike — one day's ridership tells you most of the next day's. Split such data at random and a point's own near-future lands in the training set, so the score you report is not the score you would get in deployment. It is optimistic by a margin we will measure. So the split changes, the baselines change, and the first thing to derive is what makes a series predictable at all.

1.1 The brief

Forecast the number of people who will board rail and bus tomorrow, from twenty years of daily boarding totals. The decisions it changes — how many train sets leave the depot, how many drivers are rostered, whether a spare crew is held — are made the evening before, which fixes the horizon at one day and fixes when the forecast must exist: *before the day it describes*.

The two errors are not equal, and we choose a symmetric metric anyway

Forecast too high and empty vehicles run: fuel, crew hours, a line in the budget. Measurable and survivable. Forecast too low and passengers are left on platforms at peak: the cost is political and appears in no table you will be shown.
We say that out loud now rather than discovering it later.

Framed: supervised, because the day arrives and then we know; regression, because the target is a count; batch and offline, because there is one row per day and the whole record fits in memory. The label is free — nobody annotates anything, because the next day's value is the label for today's window, which Chapter 13 calls the trick that makes a series into a supervised dataset. But the rows are not exchangeable: neighbouring days are almost the same measurement made twice.

1.2 Four lines of tidying, one of which matters

The data is fetched by a function rather than a download, the same shape as the loader in the first application. The file ships with 7,701 rows and 7,639 distinct ones: sixty-two days appear twice, identically, and nothing warns you. Ask what was dropped: here the answer is 62, which you only know because you counted.

One categorical column, `day_type`: 5,336 ordinary working days, 1,216 marked U (the second

weekend day and public holidays) and 1,087 marked A. This column is unusual and worth pausing on: *we know it in advance*. The next day's type is on the calendar today, so it is a legitimate input to a forecast of that day. Most features are not like that.

Plotting twenty-one years shows three things, none of which is what we will model: bus falling while rail rose and then fell, an annual wobble of about a tenth of the level, and March 2020 as a *cliff* rather than a dip. A model fitted across that cliff is being asked to describe two different cities, so we model 2016 to 31 May 2019 — 1,247 days of one regime. Everything after is set aside, not because it is uninteresting but because we cannot both use it and claim to have been surprised by it. Data snooping applies to time as well as to rows.

Zooming in until individual days are visible shows a seven-day cycle so strong it dominates the plot: working days near the top of the range, the two weekend days at roughly half. A repeating pattern with a fixed period is called **seasonality** — the period here is a week, not a year, and the word has nothing to do with the seasons. There is also one trough in the wrong place. Ask the data rather than the calendar: the three days around it are marked A, U, U — a public holiday extending the weekend, which the column we were given already knows. *Every anomaly should end either in an explanation or in a note that you could not find one. Not in silence.*

Seasonality and trend are different things

Seasonality is a pattern that repeats with a fixed period: here seven days, strongly, and twelve months, weakly. A **trend** is a slow movement of the level itself, with no period: here, rail rising to about 2015 and then falling. A moving average of the right width removes the seasonality and leaves the trend.

2 Derivation: stationarity, differencing and autocorrelation

Every model in this course assumes the data it was trained on and the data it will meet are the same kind of thing. For a series, what does that assumption say — and does ours satisfy it?

Definition 2.1 (Weak stationarity)**.** $\{y_t\}$ *is weakly stationary if* $\mathbb{E}[y_t] = \mu$ *is constant,* $\mathrm{Var}(y_t) = \sigma^2$ *is constant and finite, and* $\mathrm{Cov}(y_t, y_{t+k}) = \gamma_k$ *depends on* k *only.*

Only the first two moments, and only the requirement that the covariance *forgets when and remembers only how far apart*.

Why a model depends on it

Fitting minimises an average loss over the training rows, and we then quote that as an estimate of the loss on future rows. That step is a *claim*: the future rows come from the same distribution. For rows indexed by time, "the same distribution" is exactly what weak stationarity asserts about the first two moments.
If μ drifts, a model fitted on the past is systematically wrong on the future — not noisily wrong, *biased*. That is the difference between an error that averages out and one that accumulates.

2.1 Differencing

With $(\nabla y)_t = y_t - y_{t-1}$, applied to a sampled function ∇ is a discrete derivative. One value is lost at the start each time; count those losses, because `diff` returns `NaN` there and says nothing.

A linear series becomes a constant, and the constant is the slope — and a constant mean is the first condition of weak stationarity.

Seasonal differencing, $(\nabla_s y)_t = y_t - y_{t-s}$, removes a periodic component *exactly*: if $y_t = c_t + r_t$ with $c_t = c_{t-s}$, then $(\nabla_s y)_t = r_t - r_{t-s}$, with no approximation and no fitted parameter. On our series, the weekly cycle is what ∇_7 removes.

2.2 Autocorrelation, and when differencing helps

For a weakly stationary series, $\rho_k = \gamma_k/\gamma_0 \in [-1, 1]$ is the correlation of the series with itself k steps later; $\rho_0 = 1$ by construction, and the whole function $k \mapsto \rho_k$ is the **autocorrelation function**. Note it is defined *only* under stationarity: without it the covariance depends on t as well as k and there is no single number to plot. Our series is not stationary, so the ρ_k quoted for it below are *sample* autocorrelations: the same formula applied to the 1,247 days of the pool, with one mean and one variance for the whole stretch. That is a descriptive statistic of these days, not an estimate of a population ρ_k, and the notebook checks that the identity below, fed with it, still predicts the spread of each difference.

The variance of a lag-k difference

$$\begin{aligned}\operatorname{Var}(y_t - y_{t-k}) &= \operatorname{Var}(y_t) + \operatorname{Var}(y_{t-k}) - 2\operatorname{Cov}(y_t, y_{t-k}) \\ &= 2\sigma^2(1 - \rho_k).\end{aligned}$$

So differencing at lag k reduces the variance if and only if $\rho_k > \frac{1}{2}$.

On our series $\rho_1 = 0.419$ is below a half, while $\rho_7 = 0.831$ is well above it: the first difference *must* make the series more variable and the seventh *must* make it less. The notebook predicts both spreads from the identity before it measures them.

Why copying last week should be hard to beat

"Copy the value from seven days ago" has error $\hat{y}_t - y_t = -(\nabla_7 y)_t$ — its error *is* the seasonal difference. So its error variance is $2\sigma^2(1 - \rho_7)$, which at $\rho_7 = 0.831$ is $0.34\,\sigma^2$.
It should be hard to beat for one reason and one reason only: ρ_7 is large. It is not a lucky heuristic — it is the autocorrelation function, read off at a single lag.

3 Baselines, and the metric

We predict $\hat{y}_{t+1} = f(y_{t-55}, \ldots, y_t)$: given the last eight weeks of daily boardings up to and including today, the next day. Three metrics:

$$\text{MAE} = \frac{1}{m}\sum_i |\hat{y}_i - y_i|, \qquad \text{RMSE} = \sqrt{\frac{1}{m}\sum_i (\hat{y}_i - y_i)^2}, \qquad \text{MAPE} = \frac{1}{m}\sum_i \frac{|\hat{y}_i - y_i|}{y_i}.$$

RMSE punishes a large miss quadratically, MAE does not, and MAPE is relative and therefore unit-free. Choosing a metric is choosing what to care about: the book's rule is that if the project suffers quadratically more from large errors, prefer the squared metric. Ours does not. MAE is the choice, because the units are *passengers* and the operations team can act on "we were out by forty thousand boardings"; because the worst days are holidays the calendar already tells us about, and we do not want the metric dominated by days you can look up; and because RMSE would make one bad holiday worth more than a whole ordinary month. MAPE is reported beside it, never as the objective — it divides by the truth, so it is most sensitive exactly where the model is weakest, and a metric like that makes every improvement look like noise.

Three baselines, all scored on exactly the rows the models will be scored on: predict a constant, copy the day before, copy last week. None of these is machine learning; all of them are systems a transit authority could deploy on the first day.

Failure condition — a shuffled split on ordered rows

Shuffle the rows before splitting and the score improves, because every test point is surrounded in time by its own neighbours. Nothing warns. What is reported as a forecast is a measurement of interpolation. (§5 measures by how much.)

The number to beat

Copying the value from seven days ago gives an MAE of **55,399** over 1,191 days — 12.0% of a typical day, and 2.4 times better than copying the previous day. The seven-day cycle is worth more than recency. On the three months the book plots it is better still: 42,143 for rail (9.0%) and 43,916 for bus (8.3%).
It needs no training, no maintenance, no monitoring and no data scientist, and it fails only on days that appear on a public calendar. A transit authority could put it into service in an afternoon.

3.1 Turning one series into a table

The dataset class is nine lines, and the subtlety is entirely in `__len__` and in the single index that is both the end of the window and the position of the label. Every row is the row above it shifted by one day, so the rows overlap almost completely.

Eight weeks is a whole number of seven-day cycles, so every position sees the same phase of the week, and the 1,247-day pool yields 1,191 rows.

4 The linear model

$$\hat{y}_{t+1} = b + \sum_{k=0}^{55} w_k \, y_{t-k}.$$

Fifty-six weights and a bias — and if the model learns $w_6 = 1$ with every other weight zero, it has reinvented copying last week. *The linear model contains the baseline*, so it cannot do worse unless we fit it badly.

Under shuffled five-fold cross-validation it scores 44,925 against 55,399: 18.9% better, with folds

from 39,538 to 51,096, and over twenty different shuffles the mean moves by about 300 — so the improvement is far larger than the noise in the estimate.

Plot the fifty-six weights against their lag and the multiples of seven stand out, every one of them positive. *It rediscovered the seven-day cycle from the data. Nobody told it that a week has seven days.*

5 The protocol, and what it is worth

That 18.9% came from a shuffled split. The upper protocol — held-out days scattered among training days — asks the model to fill a gap; a single cut, all earlier days fitted and all later days scored, asks it to forecast. Only one of those is the job. Cross-validation estimates the risk

$$R(f) = \mathbb{E}_{(X,Y)}\big[\ell\big(f_{\mathcal{D}}(X), Y\big)\big], \qquad (X, Y) \perp \mathcal{D},$$

and it is unbiased for R under two conditions: the held-out rows are independent of the training rows, and they are drawn from the distribution the model will meet. A shuffled split of a time series satisfies neither.

Condition 1 fails and ρ_k says by how much: the covariance is $0.83\sigma^2$ at lag 7 and never near zero for any lag inside the window. A model fitted on rows *surrounding* a held-out row faces a smaller conditional variance than one fitted only on rows before it, so the estimate is biased downwards — optimistic in the overwhelming majority of cases. Not "always": that would be a theorem, and the conditional-variance argument is a heuristic rather than a bound. What we have is a measurement: on this series, 9.0% after matching training size, test size and the baseline.

Condition 2 fails and it is not subtle. In deployment every training day precedes every day being forecast; under a shuffled split, on average half the training days come *after* the day being scored. The model is not forecasting. It is being handed the shape of the series around the point it is asked about, and interpolating — a different and much easier problem, and nobody is paying for it.

So take the split the system will actually face: fit on everything up to the end of 2018 and forecast the five months that follow, one day at a time, never seeing any of them.

Model	Random split	Forecasting forward	Gap
Linear, 56 lags	44,925	56,446	+25.6%

The same model's error rises by a quarter once it has to forecast forward — not all of it the split, as the rest of this section shows.

5.1 How much of the gap is the split?

A single cut gives one score. A **rolling-origin backtest** refits at several origins, each time on the window before the origin, and scores the block immediately after it: several out-of-sample scores, and a spread you could not get from a single cut.

Changing the protocol also changes *which rows* are scored, and some rows are harder than others. Copying last week needs no fitting, so its error measures only how difficult a set of days is: on

the five months after the cut it is 17% worse than on the whole pool, because that period contains the winter holidays. So put the baseline beside every protocol: the linear model's margin over copying last week falls from 18.9% under the random split to 12.9% forecasting forward, with a rolling backtest in between — a comparison immune to the difficulty of the period.

Changing the protocol also changes how much the model trains on. Hold that fixed too — 700 training rows for every origin and every random draw, 98 days scored each time — so that only the *arrangement* of the rows differs:

Protocol	Model	Naive	Ratio
random draws of 98 rows, 20 of them	48,093	61,461	0.782
rolling origin, 5 origins	47,212	55,371	0.853

Each is scored against copying last week *on its own rows*, which cancels the difficulty of the period. The random split reports the model as 9.0% more skilful than it is.

The honest verdict, in three numbers

25.6% is the raw gap between the linear model's random-split score and its forward forecast. That is what you would have been wrong by, and it is real.
17% of it is the forecast period genuinely being harder — the baseline moves too.
7.4% is the protocol itself: how much the random split overstates the linear model's skill over copying last week, with each protocol's baseline computed on its own rows. It survives every control we could apply: match the training size and the test size as well, and the overstatement is 9.0%. The recurrent network of Lecture 16 will be measured the same way — forward in time, against copy-last-week on the identical rows — so its number can be read against this one.

Put in the terms a stakeholder hears: we told them our linear model was 18.9% better than doing nothing, and it is 12.9% better. A third of the case for building it was an artefact of the evaluation.

5.2 Purging and embargo

Even a clean cut leaks a little: the last training window overlaps the first scored day's history. The repair is to leave a gap — **purging** the training rows whose windows overlap the scored ones, and adding an **embargo** of a window length after the cut. On this data the effect is small, because the overlap is 56 rows out of 1,040; on a dataset where windows are long relative to the record, it is not. We name it but do not use it. Purging and embargo are standard practice in financial backtesting and are outside the book, so not examinable; the idea — that adjacency itself is a channel — is examinable, because we derived it.

6 The notebook

What to change

Shuffle the rows before splitting and re-run the evaluation. The score improves, and nothing warns you. That improvement is the whole argument of the second half of this lecture, produced by you rather than quoted at you.

7 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 16's deck and in the exercise book.

1. For a stationary series, $\mathrm{Var}(X_t - X_{t-h}) = 2\gamma(0)(1 - \rho(h))$. State the condition on $\rho(h)$ under which differencing *increases* the variance. [4]
2. Explain why the forecast "copy the value from seven days ago" is hard to beat on daily ridership, in terms of the autocorrelation function. [4]
3. State why MAPE is the wrong metric for transit ridership, using a specific day as the example. [4]
4. A cross-validated forecast uses `KFold(shuffle=True)` and reports a good score. Name the two conditions the split violates. [5]
5. A dataset class turns a pool of 1,247 daily values into windows of 56 days, each labelled with the day that follows it. How many rows does it yield, and what goes wrong if the label is taken one index earlier? [4]

Summary

Weak stationarity asks for a constant mean, a constant finite variance and a covariance depending only on the lag — and models depend on it because they quote a training average as an estimate of a future one. Our series does not have it. One difference turns a linear trend into a constant, and the seasonal difference ∇_7 cancels the weekly cycle exactly.

$\mathrm{Var}(y_t - y_{t-k}) = 2\sigma^2(1 - \rho_k)$, so differencing helps if and only if $\rho_k > \frac{1}{2}$. The naive forecast's error *is* the seasonal difference, so its strength is $\rho_7 = 0.831$ and nothing else — which predicted, before it was measured, that copying last week would be hard to beat. A linear model on 56 lags contains that baseline, and found the week on its own.

Then the lecture's real subject. A shuffled split violates both conditions cross-validation needs, and it reported the linear model as 18.9% better than nothing when it is 12.9% better. Of its raw 25.6% gap, 17% is the forecast period genuinely being harder; the protocol itself is 7.4% with the baseline recomputed on identical rows, and 9.0% once training size and test size are matched as well.